Relatório do Sistema de Gerenciamento de Mercado

Aplicação web em Django | Gestão de estoque, caixa e ponto de venda

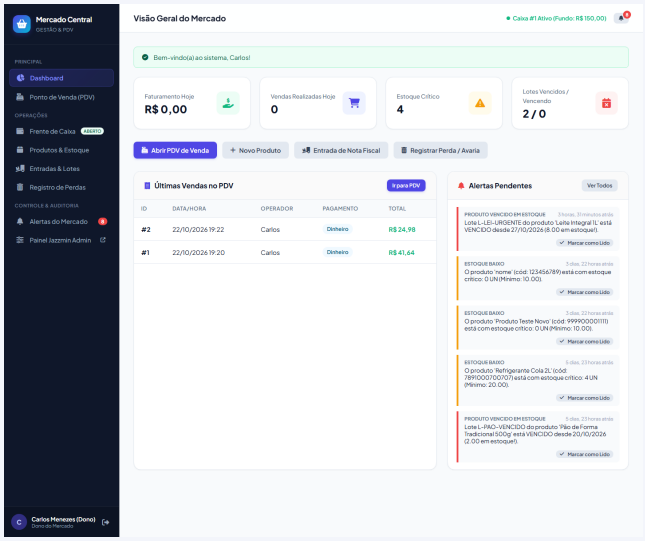
1. Objetivo e implementação

O projeto foi pensado para pequenos comércios que precisam controlar vendas, estoque e validade sem depender de um sistema comercial caro. A aplicação reúne essas rotinas em uma interface web e organiza o código no padrão BCE (Boundary–Control–Entity).

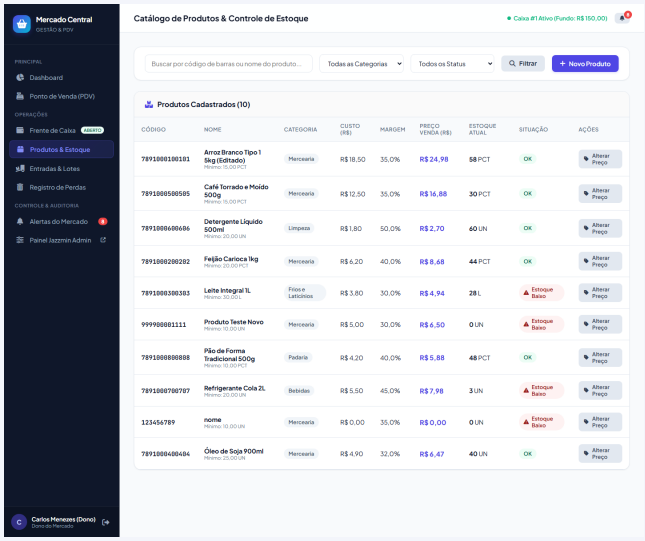
Funcionalidades implementadas: acesso por perfis (dono, gerente, estoquista e operador); cadastro e precificação de produtos; entradas registradas com fornecedor, número de nota e lotes; abertura e fechamento de caixa com conferência; PDV com busca e pagamentos; registro de perdas; e alertas de estoque baixo e vencimento.

Na venda, o estoque é baixado por lote com prioridade para o vencimento mais próximo (FEFO). As movimentações e itens de venda permitem acompanhar o histórico operacional.

2. Telas da aplicação



Painel: vendas, estoque crítico e alertas



Produtos: preços, quantidades e situação do estoque

Qualidades: interface visual consistente, perfis de acesso, regras de negócio separadas das telas e controle por lotes. O painel dá visibilidade rápida de vendas e situações que precisam de atenção.

3. Modelagem e escopo

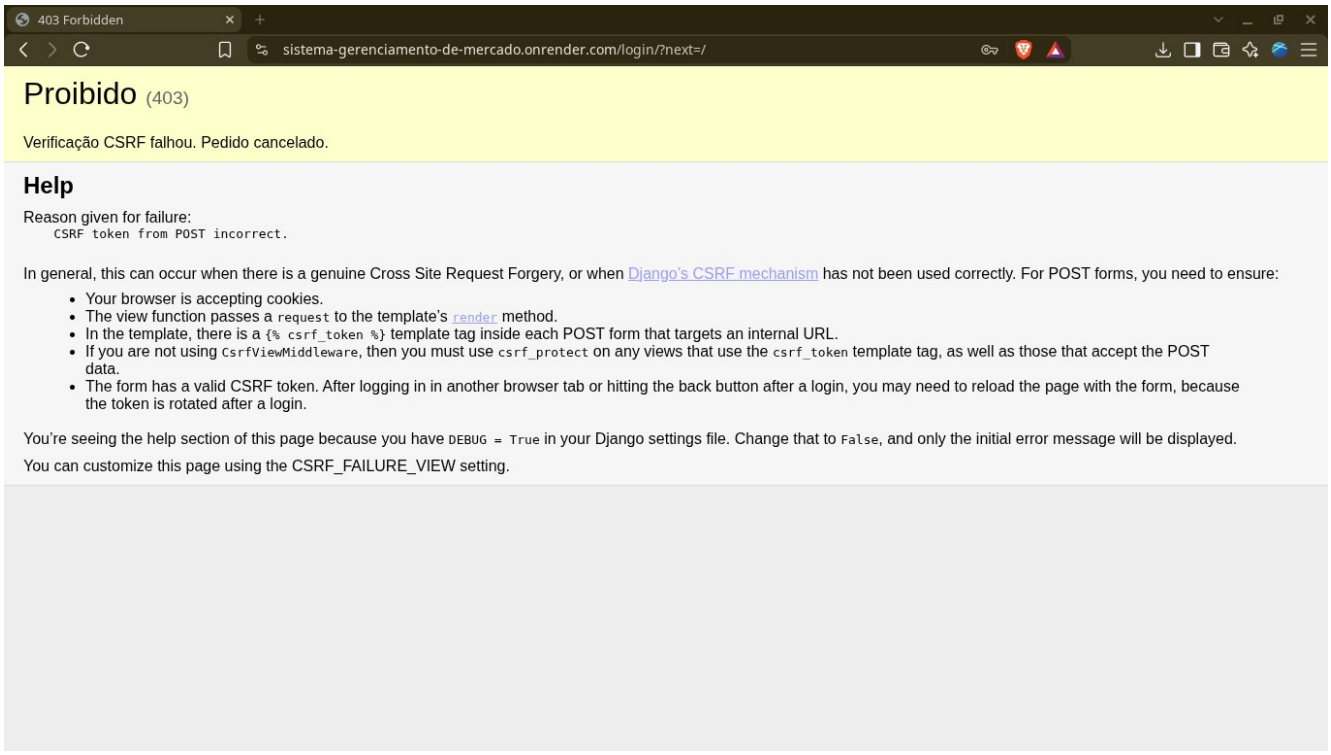
A modelagem BCE foi concretizada em entidades Django (por exemplo, Produto, Lote, EntradaMercadoria, Caixa, Venda, ItemVenda, Perda, MovimentacaoEstoque e Alerta), regras nos controladores de *services.py* e interação nas views e templates. O detalhamento de lotes, itens de venda, movimentações e alertas dá suporte à rastreabilidade do estoque e da operação.

Ficou de fora: integração com uma API autorizada para consultar ou emitir notas fiscais eletrônicas legítimas. O sistema registra informações de entrada e nota fornecidas pelo usuário, mas isso não equivale a validar ou emitir documento fiscal. Também não foram implementados gestão de múltiplas filiais nem relatórios avançados com exportação.

4. Hospedagem e dificuldades

A hospedagem atual do projeto é o Render; o endereço de acesso é sistema-gerenciamento-de-mercado.onrender.com. O endereço do PythonAnywhere não é mais utilizado. A publicação no Render usa Gunicorn como servidor da aplicação e WhiteNoise para arquivos estáticos; o guia também prevê collectstatic e migrações. O projeto está configurado com SQLite, por isso uma implantação de produção precisa considerar persistência dos dados.

A principal dificuldade foi escolher e configurar um servidor, incluindo caminhos de páginas e arquivos CSS/estáticos. Houve páginas que apresentavam erro em vez do conteúdo esperado; a equipe suspeitou de imports ou paths. A captura abaixo documenta também um erro 403 do Render por falha na verificação CSRF (token POST incorreto). Esse erro confirma um problema de implantação/formulário, mas não prova que CSS ou paths fossem a causa dos outros erros. Na verificação mais recente, o endereço do Render também não respondeu normalmente (retorno 503/timeout); a disponibilidade deve ser confirmada nos logs e no painel do serviço.



Evidência do teste de hospedagem: resposta 403 — falha CSRF no POST.

5. Síntese

A aplicação entrega uma base funcional para gestão de um pequeno mercado e deixa espaço para evolução. Como próximos passos, recomenda-se estabilizar o deploy no Render e a configuração CSRF/arquivos estáticos, adotar banco persistente em produção e avaliar a integração fiscal somente após obter as autorizações necessárias.

Fonte complementar: *Guia de Hospedagem para Sistema de Gerenciamento de Estoque*, material da equipe enviado para enriquecer este relatório. A hospedagem descrita neste relatório refere-se ao Render.